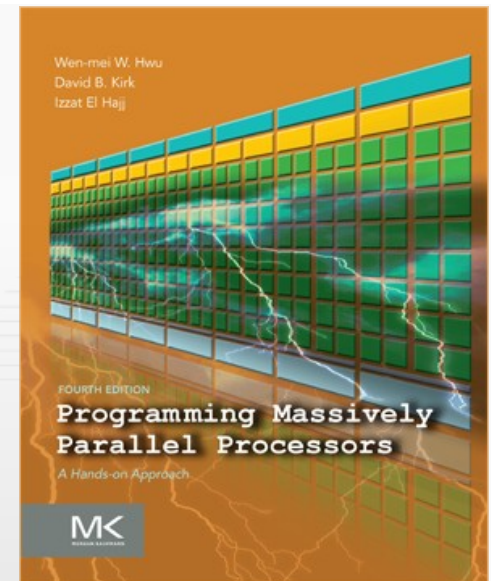


Programming Massively Parallel Processors

A Hands-on Approach

CHAPTER 14 (PART 2) > Sparse Matrix Computation



- Advantages:
 - Space efficiency: row pointers smaller than row indexes
 - Accessibility: given a row, easy to find all nonzeros
 - SpMV/CSR avoids atomics, every thread owns its output
- Disadvantage:
 - Flexibility: hard to add new elements to the matrix
 - Accessibility: given nonzero, hard to find row; given a column, hard to find all nonzeros
 - SpMV/CSR memory accesses are not coalesced
 - SpMV/CSR has control divergence

Can we design a format that avoids atomics but also improves memory coalescing and reduces control divergence?

Matrix:

1	7		
5		3	9
	2	8	
			6

Group nonzeros
by row (like CSR)

...

Column:

0	1	
0	2	3
1	2	
3		

Value:

1	7	
5	3	9
2	8	
6		

Matrix:

1	7		
5		3	9
	2	8	
			6

...pad rows so they
all have the same
size...

Column:

0	1	*
0	2	3
1	2	*
3	*	*

Value:

1	7	*
5	3	9
2	8	*
6	*	*

Matrix:

1	7		
5		3	9
	2	8	
			6

...and store
padded array of
nonzeros in column
major order

Column:

0	1	*
0	2	3
1	2	*
3	*	*

Value:

1	7	*
5	3	9
2	8	*
6	*	*

Matrix:

1	7		
5		3	9
	2	8	
			6

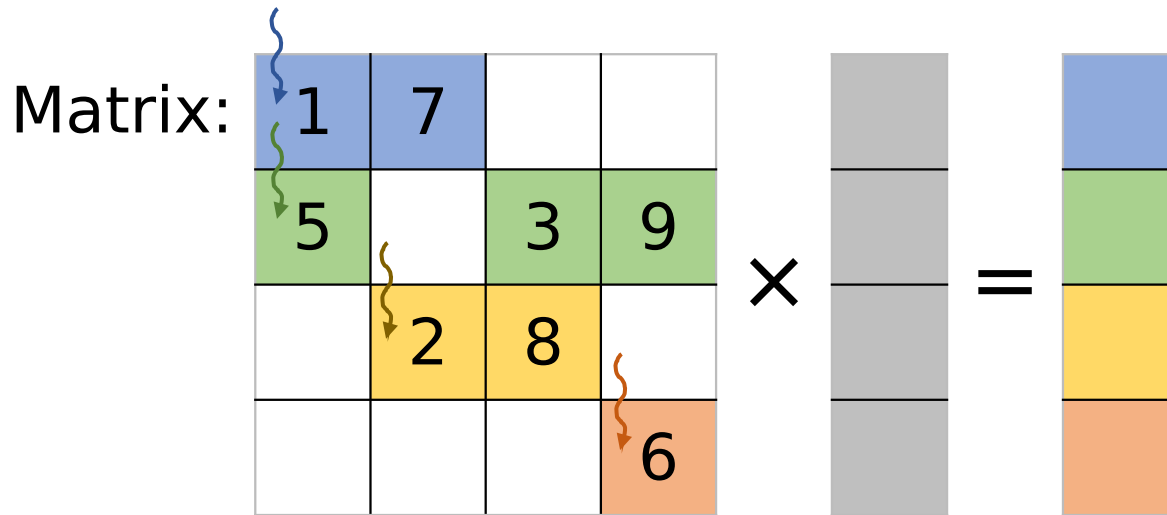
...and store
padded array of
nonzeros in column
major order

Column:

0	0	1	3	1	2	2	*	*	3	*	*
---	---	---	---	---	---	---	---	---	---	---	---

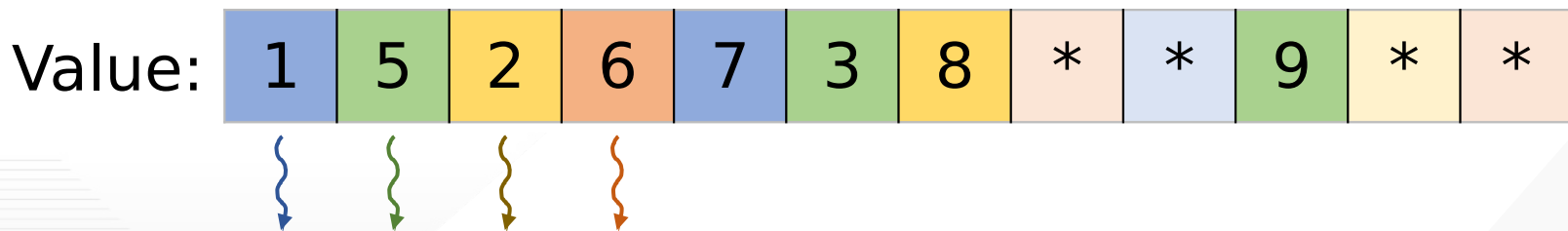
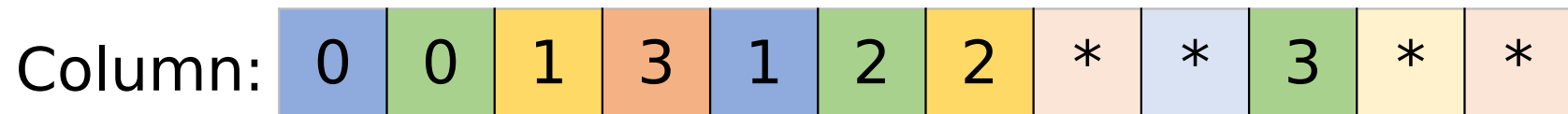
Value:

1	5	2	6	7	3	8	*	*	9	*	*
---	---	---	---	---	---	---	---	---	---	---	---



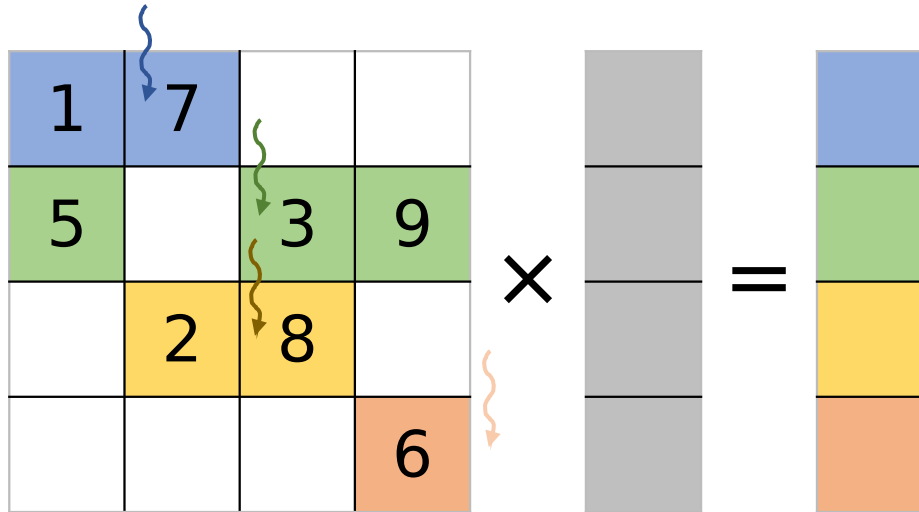
Parallelization approach:

Assign one thread to loop over each input row sequentially and update corresponding output element



Memory accesses are coalesced

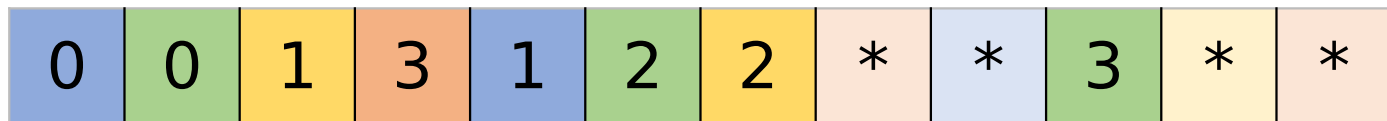
Matrix:



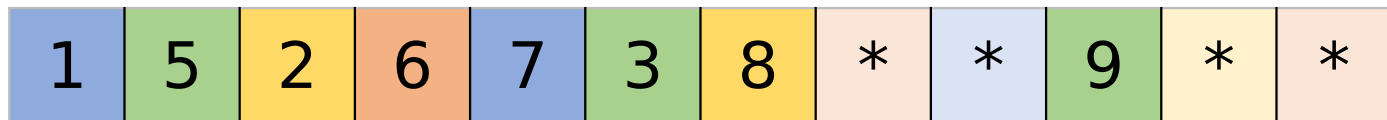
Parallelization approach:

Assign one thread to loop over each input row sequentially and update corresponding output element

Column:

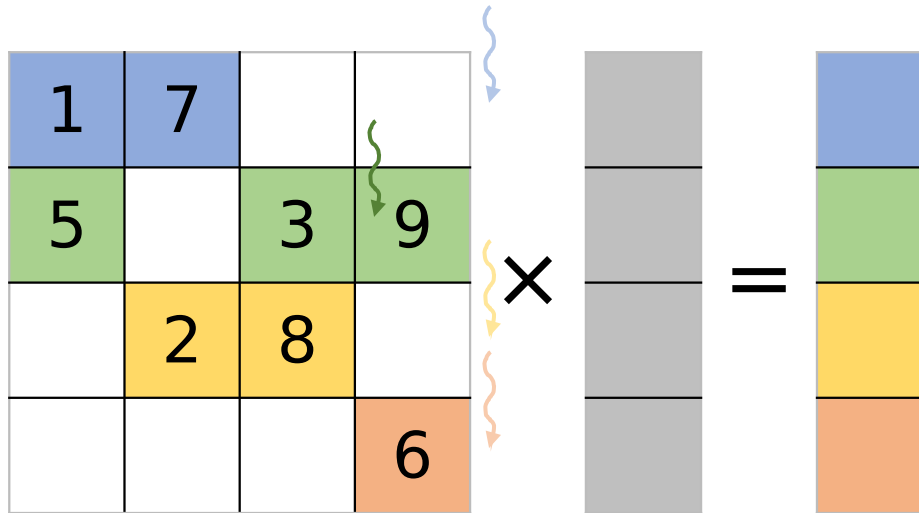


Value:



Memory accesses are coalesced

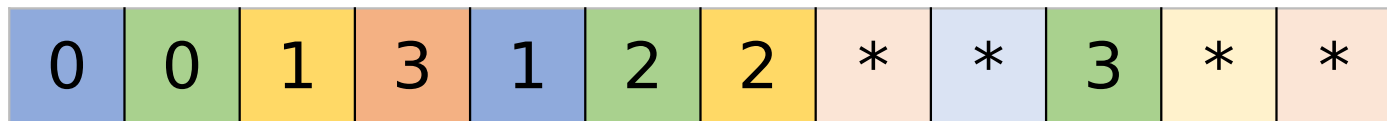
Matrix:



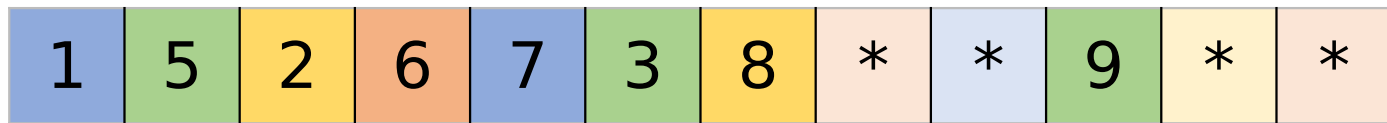
Parallelization approach:

Assign one thread to loop over each input row sequentially and update corresponding output element

Column:



Value:

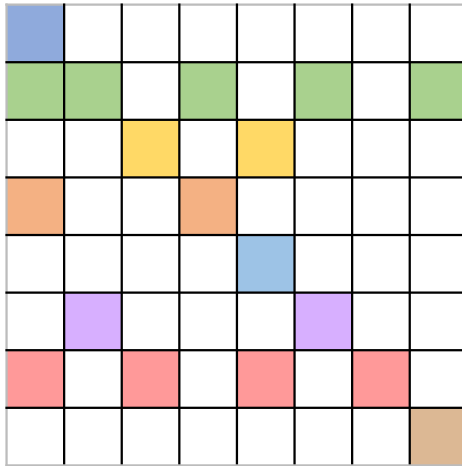


Memory accesses are coalesced

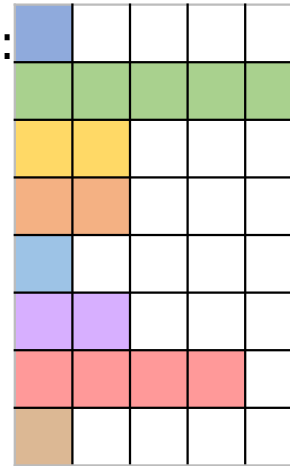
```
__global__ void spmv_ell_kernel(ELLMatrix ellMatrix, float* inVector, float* outVector) {
    unsigned int row = blockIdx.x*blockDim.x + threadIdx.x;
    if(row < ellMatrix.numRows) {
        float sum = 0.0f;
        for(unsigned int nnzIdx = 0; nnzIdx < ellMatrix.nnzPerRow[row]; ++nnzIdx) {
            unsigned int i = nnzIdx*ellMatrix.numRows + row;
            unsigned int col = ellMatrix.colIdxs[i];
            float value = ellMatrix.values[i];
            sum += inVector[col]*value;
        }
        outVector[row] = sum;
    }
}
```

- Advantages:
 - Flexibility: can add new elements as long as row not full
 - Accessibility: given a row, easy to find all nonzeros; given nonzero, easy to find row and column
 - SpMV/ELL memory accesses are coalesced
- Disadvantage:
 - Space efficiency: overhead due to padding
 - Accessibility: given a column, hard to find all nonzeros
 - SpMV/ELL has control divergence

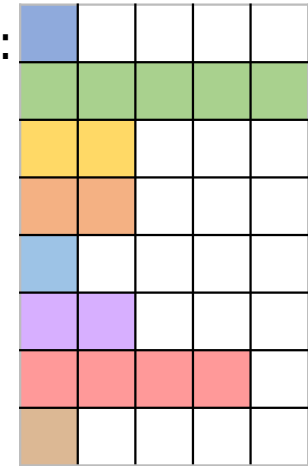
Matrix:



Column:

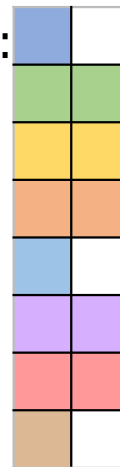


Value:

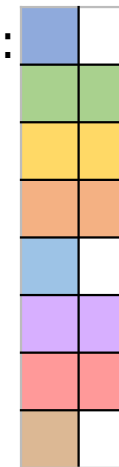


ELL Format

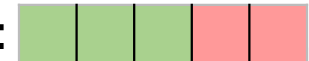
Column_{ELL}:



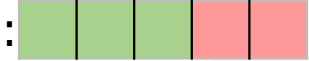
Value_{ELL}:



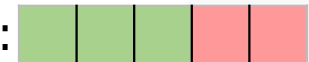
Row_{COO}:



Column_{COO}:



Value_{COO}:



Hybrid ELL + COO
(use COO for very long rows)

- Similar to ELL, with the following added benefits from using COO:
 - Space efficiency: less padding
 - Flexibility: can add new elements to any row
 - Less control divergence

Matrix:

a					
b	c		d	e	
		f		g	
h			i		j
	k			l	
m		n			o

Group nonzeros
by row (like CSR)

...

Column:

0					
0	1	3	4		
2	4				
0	3	5			
1	4				
0	2	5			

Value:

a					
b	c	d	e		
f	g				
h	i	j			
k	l				
m	n	o			

Matrix:

a					
b	c		d	e	
		f		g	
h			i		j
	k			l	
m		n			o

...sort rows by
size and
remember the
original row
index...

Column:

0	1	3	4
0	3	5	
0	2	5	
2	4		
1	4		
0			

Value:

b	c	d	e
h	i	j	
m	n	o	
f	g		
k	l		
a			

Row:

1
3
5
2
4
0

Matrix:

a					
b	c		d	e	
		f		g	
h			i		j
	k			l	
m		n			o

...store nonzeros in
column major...

Column:

0	1	3	4
0	3	5	
0	2	5	
2	4		
1	4		
0			

Value:

b	c	d	e
h		j	
m	n	o	
f	g		
k			
a			

Row:

1
3
5
2
4
0

Matrix:

a					
b	c		d	e	
		f		g	
h			i		j
	k			l	
m		n			o

...store nonzeros in
column major...

Column:

0	0	0	2	1	0	1	3	2	4	4	3	5	5	4
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Value:

b	h	m	f	k	a	c	i	n	g	l	d	j	o	e
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Row:

1
3
5
2
4
0

Matrix:

a					
b	c		d	e	
		f		g	
h			i		j
	k			l	
m		n			o

...and remember
where the
nonzeros of each
iteration start

IterPtr:

0	6	11	14	15
---	---	----	----	----

Row:

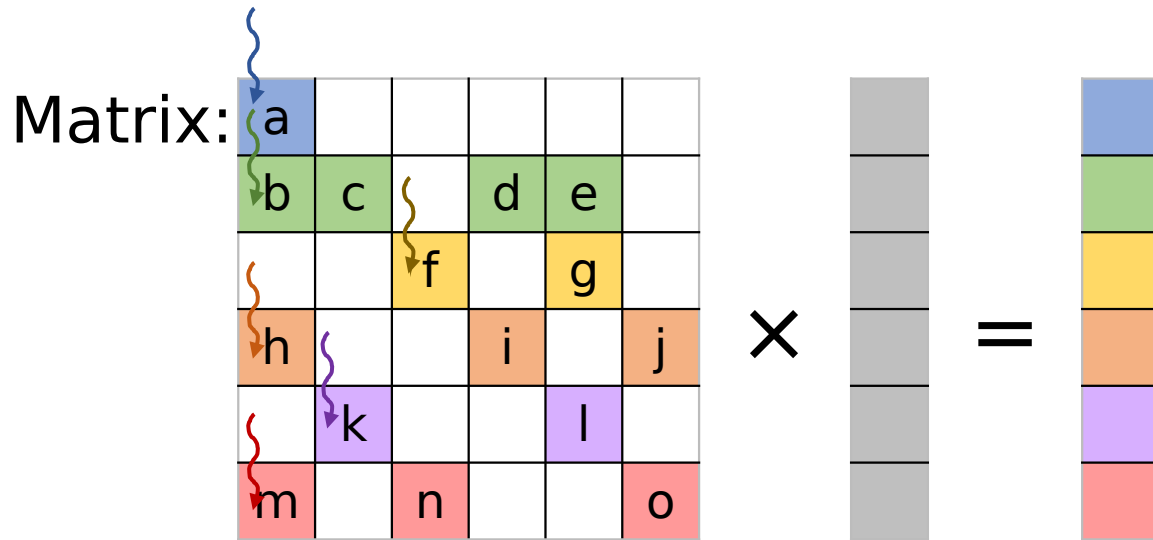
1
3
5
2
4
0

Column:

0	0	0	2	1	0	1	3	2	4	4	3	5	5	4
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Value:

b	h	m	f	k	a	c	i	n	g	l	d	j	o	e
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Parallelization approach:

Assign one thread to loop over each input row sequentially and update corresponding output element

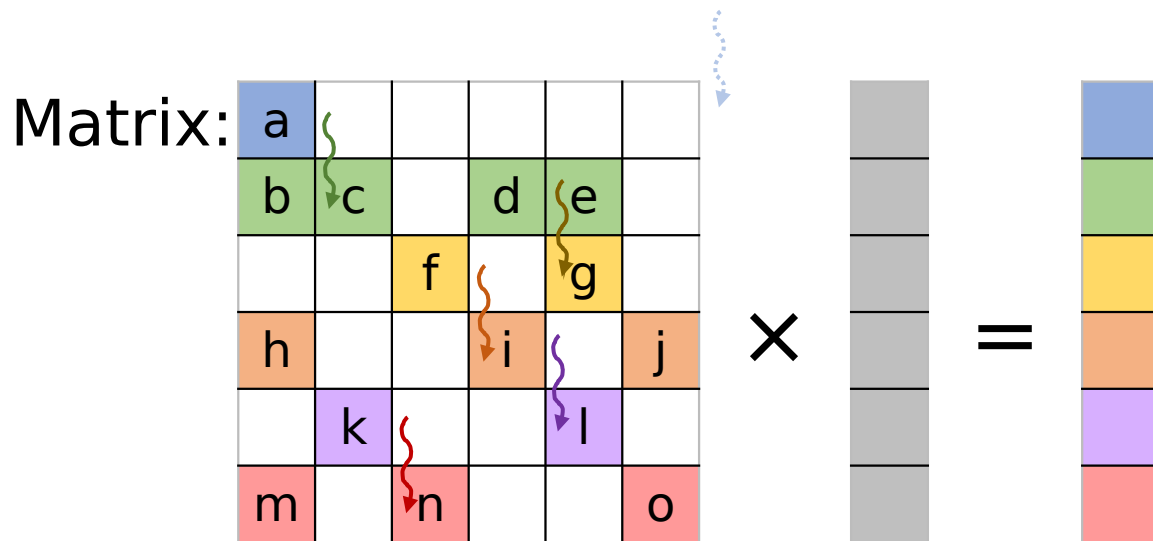
IterPtr: 0 6 11 14 15

Column: 0 0 0 2 1 0 1 3 2 4 4 3 5 5 4

Value: b h m f k a c i n g l d j o e

Memory accesses are coalesced...

Row: 1 3 5 2 4 0



Parallelization approach:

Assign one thread to loop over each input row sequentially and update corresponding output element

IterPtr: 0 6 11 14 15

Column: 0 0 0 2 1 0 1 3 2 4 4 3 5 5 4

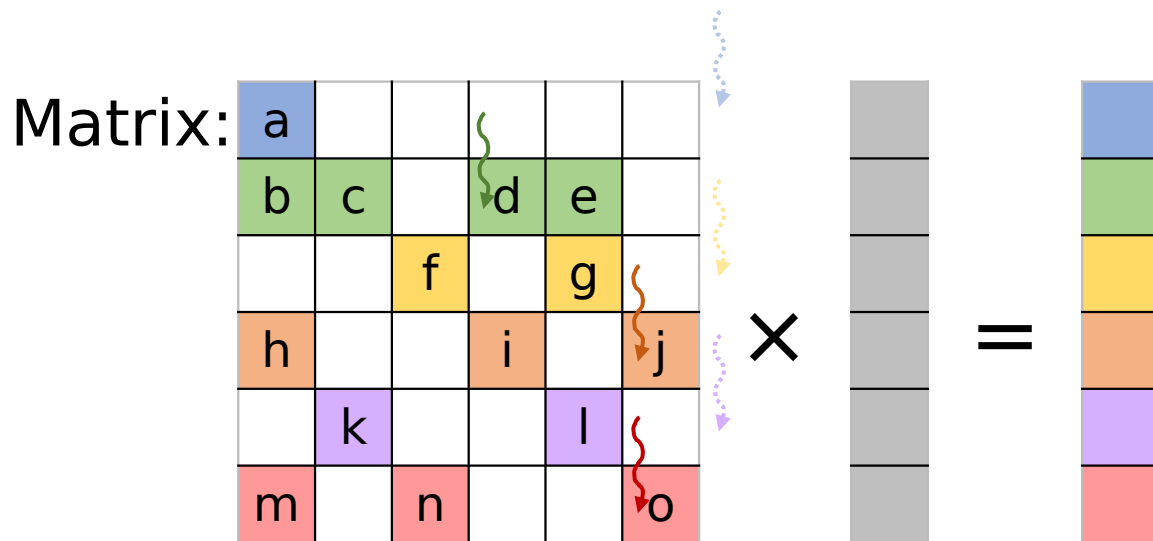
Value: b h m f k a c i n g l d j o e



Row: 1 3 5 2 4 0

Memory accesses are coalesced...

...and threads drop out from the end, minimizing control divergence



Parallelization approach:

Assign one thread to loop over each input row sequentially and update corresponding output element

IterPtr: 0 6 11 14 15

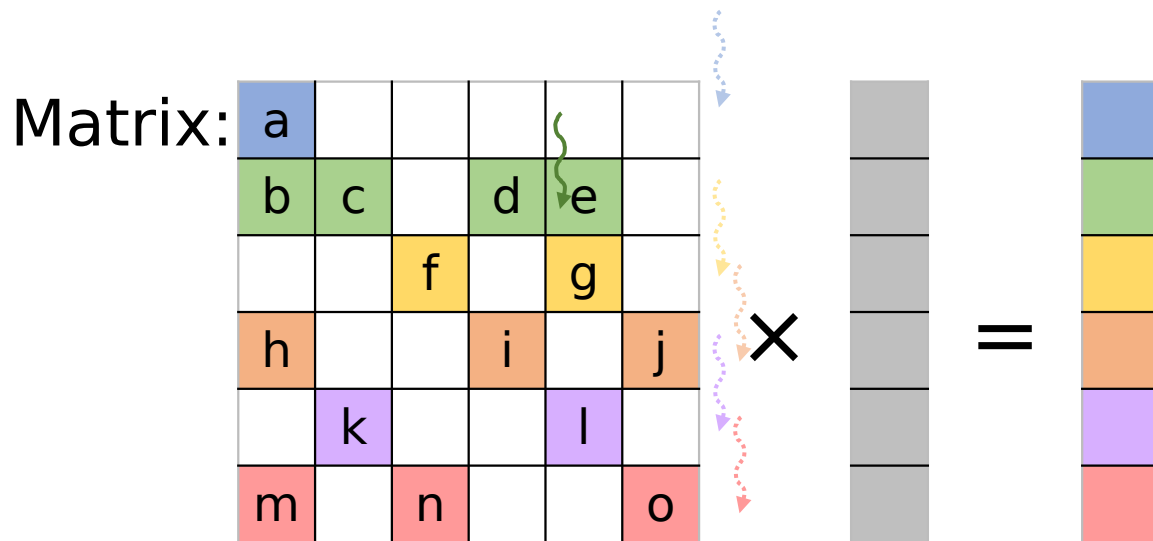
Column: 0 0 0 2 1 0 1 3 2 4 4 3 5 5 4

Value: b h m f k a c i n g l d j o e

Row: 1 3 5 2 4 0

Memory accesses are coalesced...

...and threads drop out from the end, minimizing control divergence



Parallelization approach:

Assign one thread to loop over each input row sequentially and update corresponding output element

IterPtr: 0 6 11 14 15

Column: 0 0 0 2 1 0 1 3 2 4 4 3 5 5 4

Value: b h m f k a c i n g l d j o e

Row: 1 3 5 2 4 0

Memory accesses are coalesced...

...and threads drop out from the end, minimizing control divergence

- Advantages:
 - Space efficiency: no padding
 - Accessibility: given a row, easy to find all nonzeros
 - SpMV/JDS memory accesses are coalesced
 - SpMV/JDS minimizes control divergence
- Disadvantage:
 - Flexibility: hard to add new elements to the matrix
 - Accessibility: given nonzero, hard to find row; given a column, hard to find all nonzeros

- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.